Sequence - Sequence

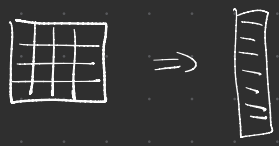
Models

Made with Goodnotes

ANN

→ ANN learns Patterns in fixed data size where order dont matter.

CNN

→ Images →  ⇒

RNN

Text → Simple RNN → Vanishing Gradient Problem

↳ LSTM
↳ GRU
} LTM

Many to many RNN

Many Inputs → Many outputs

- Encoder & Decoder (RNN Based seq 2 seq)
- (Seq 2 Seq + Attention)
- Transformer Based seq 2 seq

Made with Goodnotes

Encoders and Decoders

Problems with ANN, RNN, CNN

3 types of model

- Input → output
  - ML Algo.
  - ANN
  - CNN
- Sequence → Label
  - RNN
  - LSTM
  - GRU
- Sequence → Sequence
  - encoder & Decoder

eg

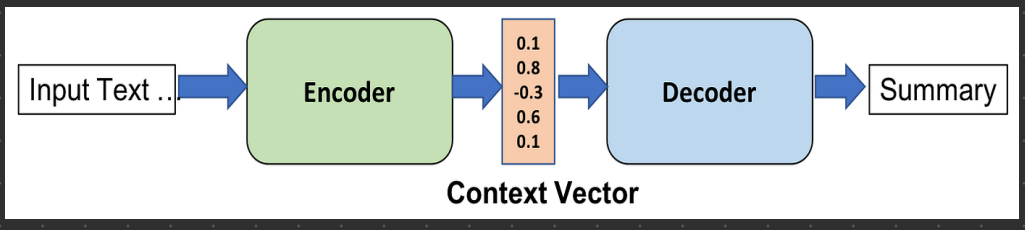
Question → Answer

English language → French language

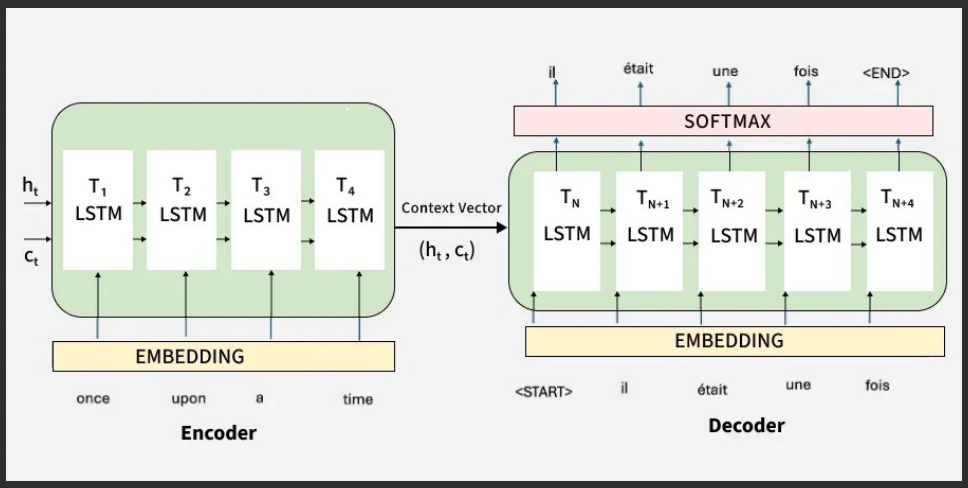
one line defination of encoder & Decoder

> An Encoder–Decoder model first understands the input sequence and then generates a new output sequence based on that understanding.

Input seq → Encoder → Context vector → Decoder → output seq



Made with Goodnotes

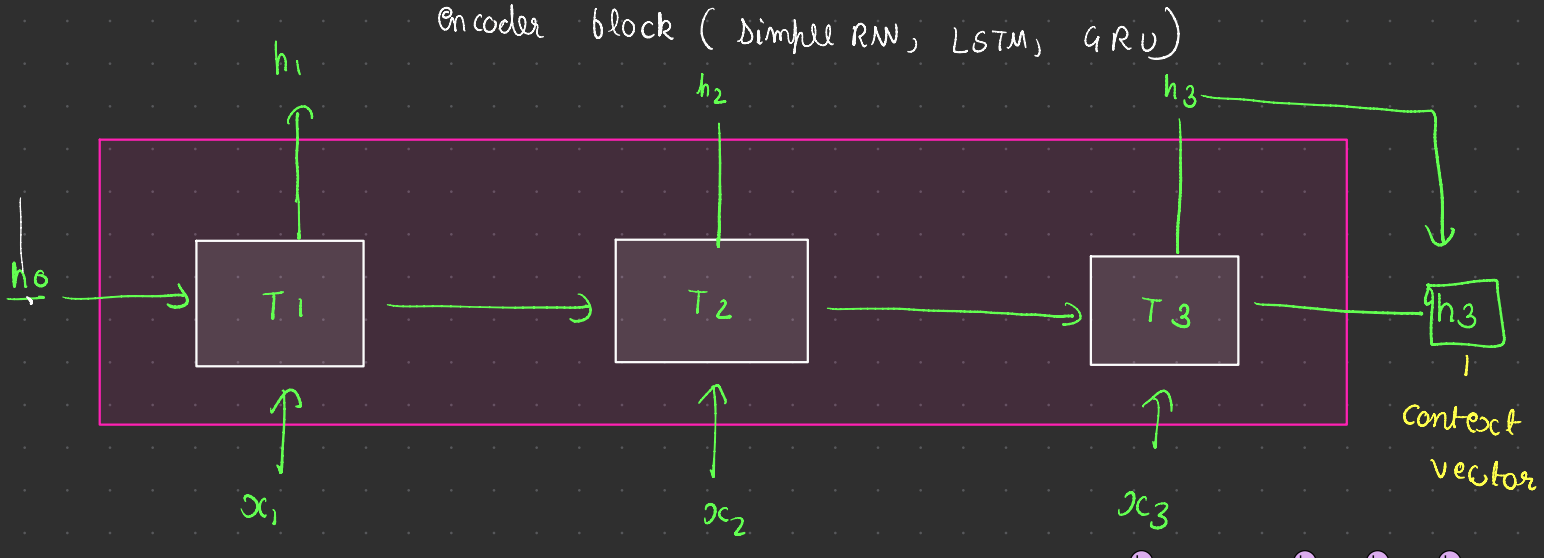


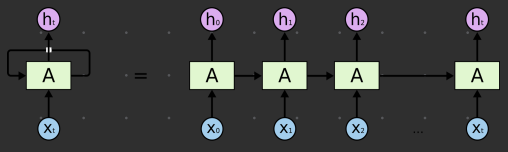
Encoders

→ converts an input sequence into a meaningful representation.

Training phase

Input	output
→ I love sherpians	<s> मुझे शेरपियन्स पसंद हैं <end>
x_1 x_2 x_3	y_1 y_2 y_3 y_4





Made with Goodnotes

Characteristics

1) Use RNN, LSTM, GRU
2) Reads input from left → Right
3) No output Generation (context vector)
4) It only understand the sentence.

Decoder

→ Generate the output seq. one element at a time.

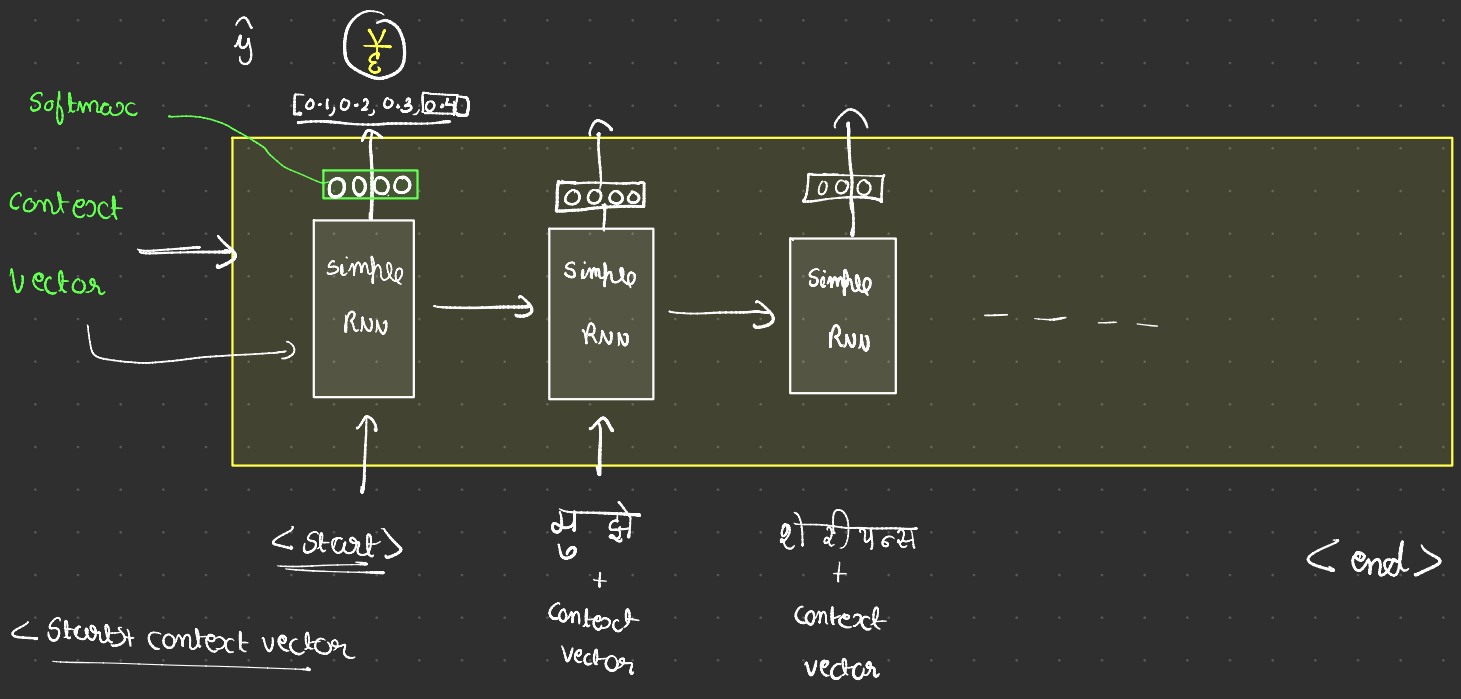
Decoder is different in training & Prediction.

Training Phase (Teacher forcing)

→ Decoder Already knows the correct output
→ At every step it is helped

Prediction.

→ Decoder does not know output
→ It uses its own previous prediction.



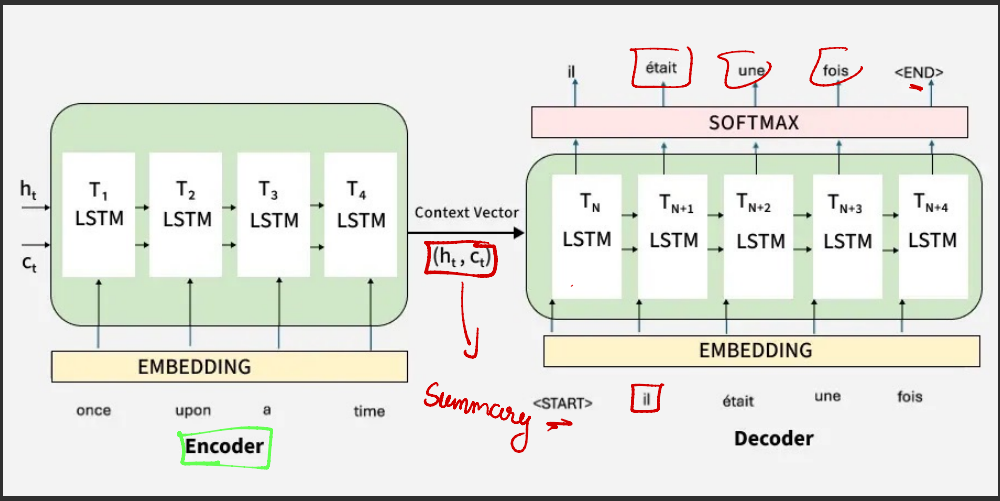
Made with Goodnotes

Characteristics.

1) Decoder is a Probability Based model.
2) softmax is always applied (training, test)
3) Decoder depends on previous word input (<start>)
4) Decoder Behaves differenty in prediction.

Problems

1) Information Bottleneck (fixed size context vector)
2) No word to word Alignment
3) Exposure Bias (Teacher forcing problem)
4) Long sentence failure

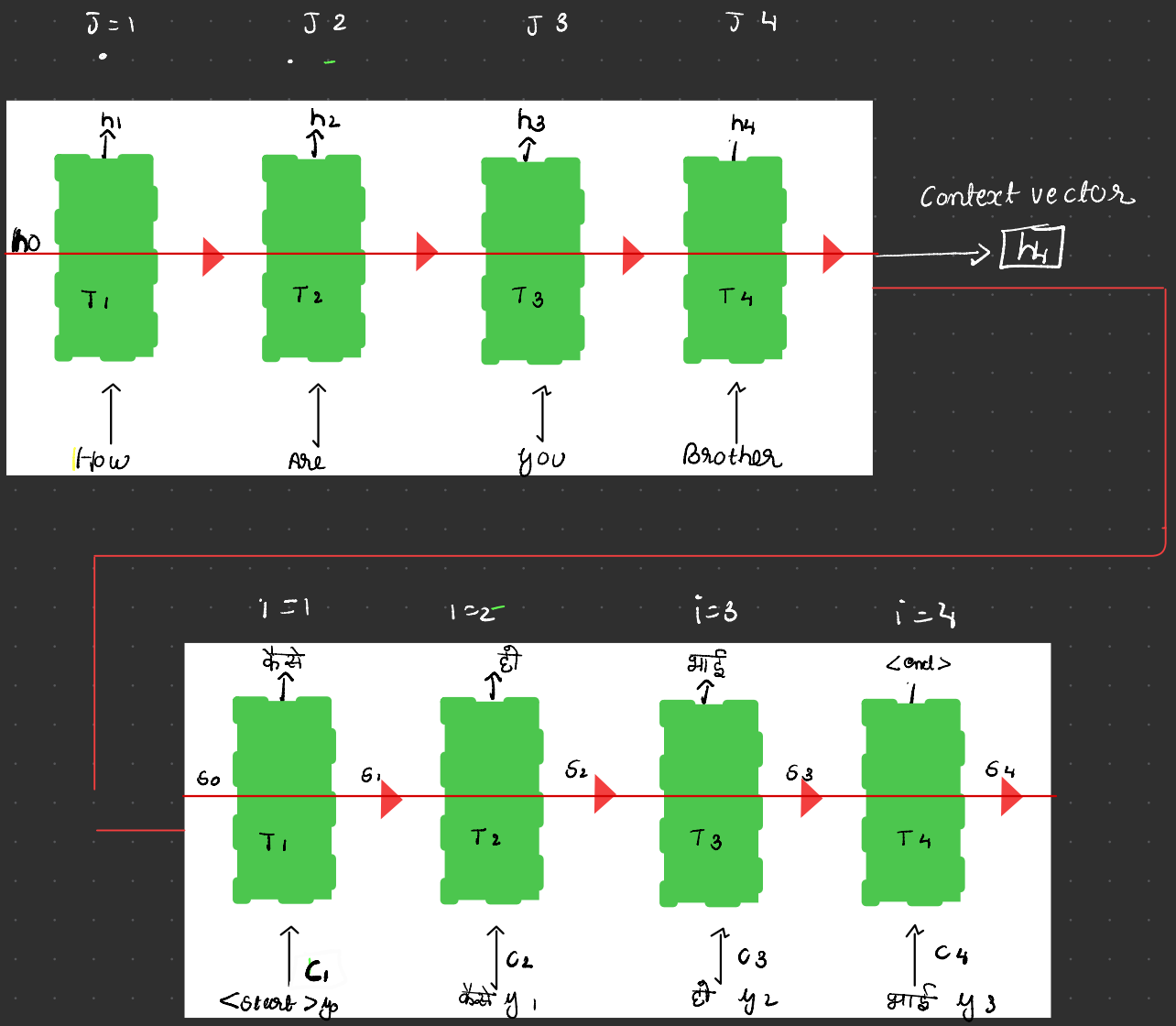


If we have a long statement

A quiet morning unfolded as the sun painted the sky in soft shades of gold and peach. People hurried along the streets with coffee cups in hand, each carrying plans, worries, and small hopes for the day ahead. Somewhere, a musician tuned a guitar, letting gentle notes drift into the air, while a child laughed without reason, reminding everyone how simple joy can be. Life rarely announces its biggest lessons; instead, they hide in ordinary moments — a kind smile, an unexpected conversation, or the courage to begin again. If you pause long enough to notice, even routine days can feel quietly extraordinary.

Made with Goodnotes

Attention Mechanism



T_1

In Decoder $\Rightarrow [S_0, y_0]$

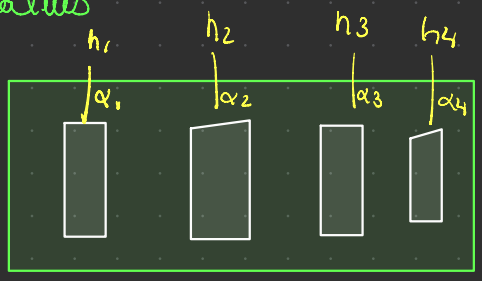
In Decoder with Attention

$[S_0, y_0]$

$+ C_1$ vector

Calculating C values

Encoder →

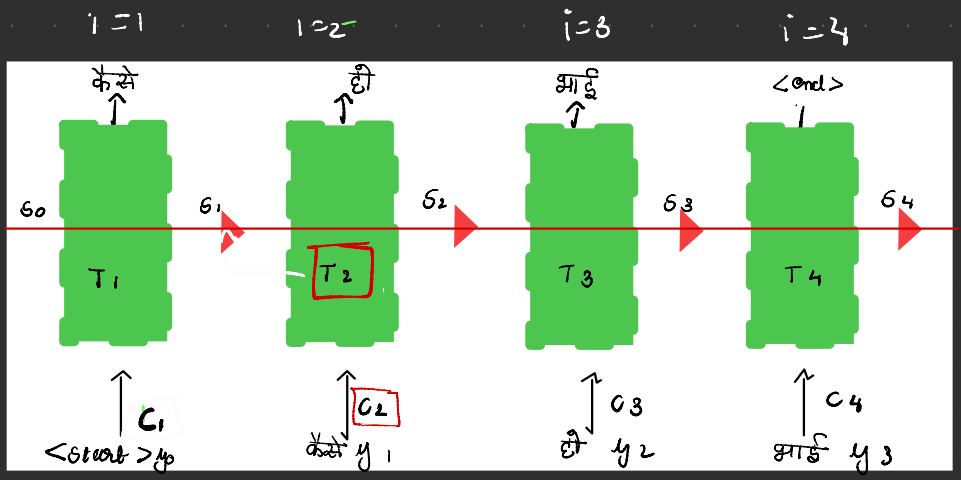


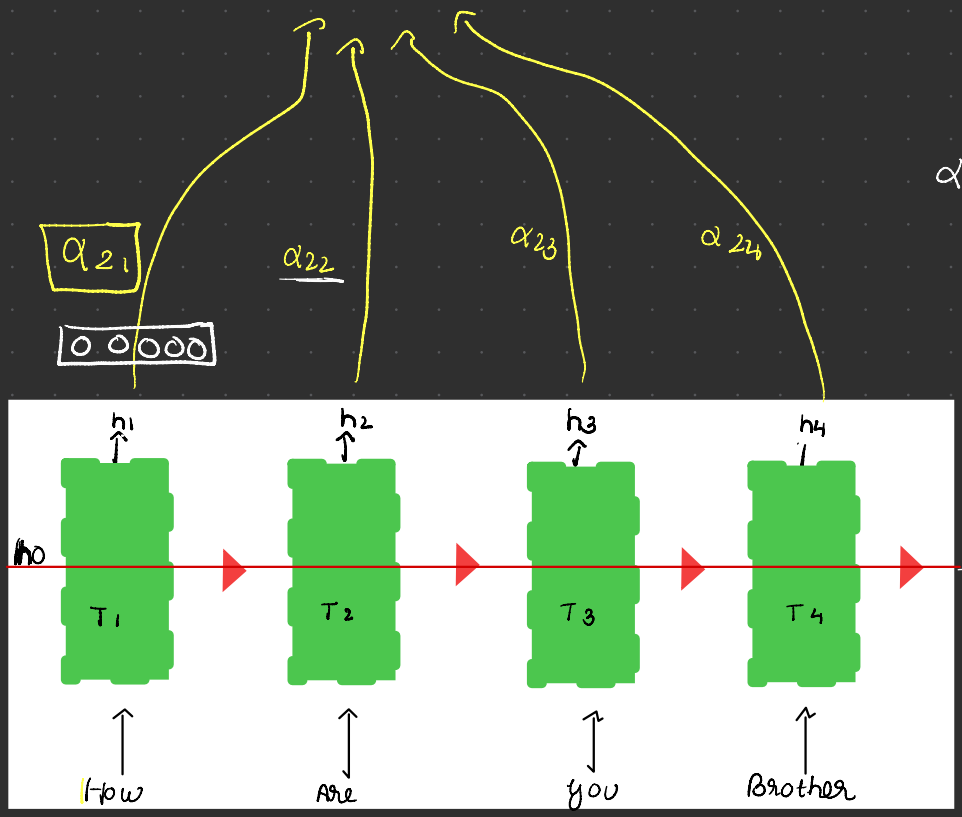
$$C_1 = \alpha_1 h_1 + \alpha_2 h_2 + \alpha_3 h_3 + \alpha_4 h_4$$

$$C_i = \sum_{j=n}^{j=1} \alpha_{i,j} \cdot h_j$$

Made with Goodnotes

Decoder





$$\alpha_{22} = f(h_2, s_1)$$

hidden state of car word through which α is connected

$$\alpha_{21} = h_1, s_1 \qquad \alpha_{21} = f(h_1, s_1)$$

Previous hidden state of decoder

$$\alpha_{ij} = f(h_j, s_{i-1})$$

Universal Approximater = ANN

Made with Goodnotes

Attention Did this	Still failed	Transformers
Let Decoder look at all encoder states	RNN is sequential	Removed RNN entirely
Easier to focus on Distant word	Signal travels still through time	Direct word to word attention
Training speed had Slight improvement.	Cannot Parallelize	Full Parallel Training
Added Alignment in Arch.	Still complex	Clean Stack design.

Made with Goodnotes

Transformers

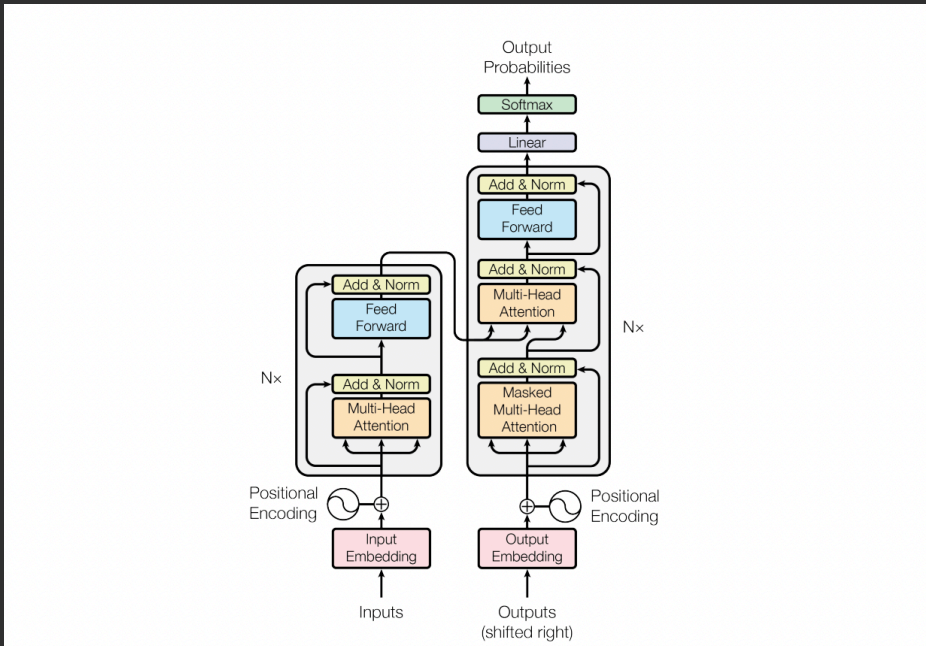
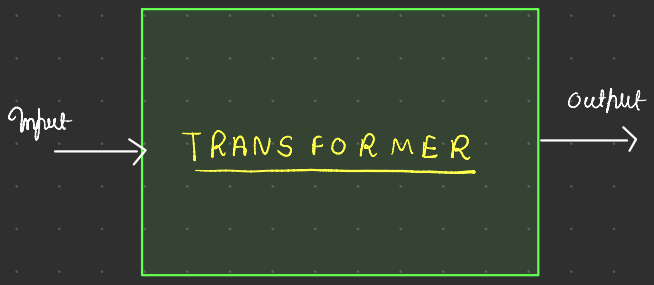


Figure 1: The Transformer - model architecture.



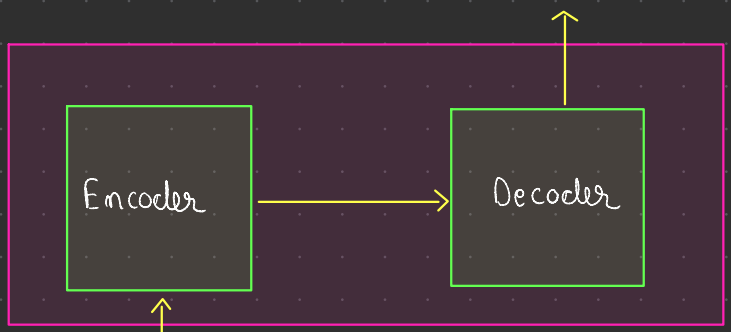
text to text ✓

Image to text ✓

text to image ✓

All the LLM's are because of transformers

Seq to Seq



कैसे हो आप

How are you?

Encoder and decoder Based model

Made with Goodnotes

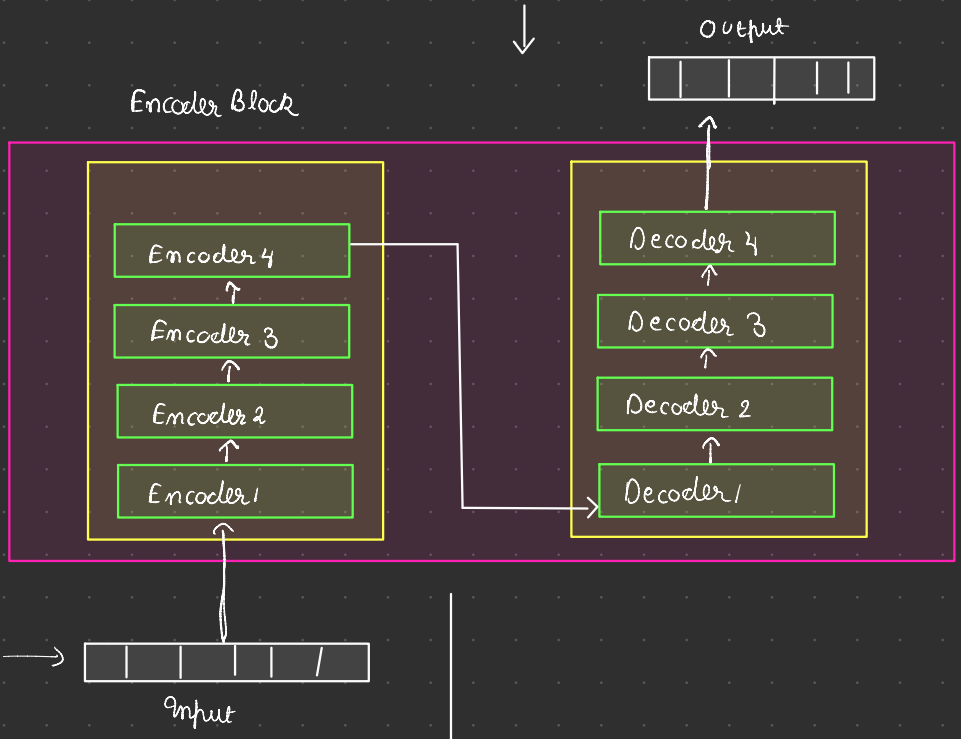
we have multiple stacked encoders and Decoders.

Lower layers → Capturing basic Relationship.

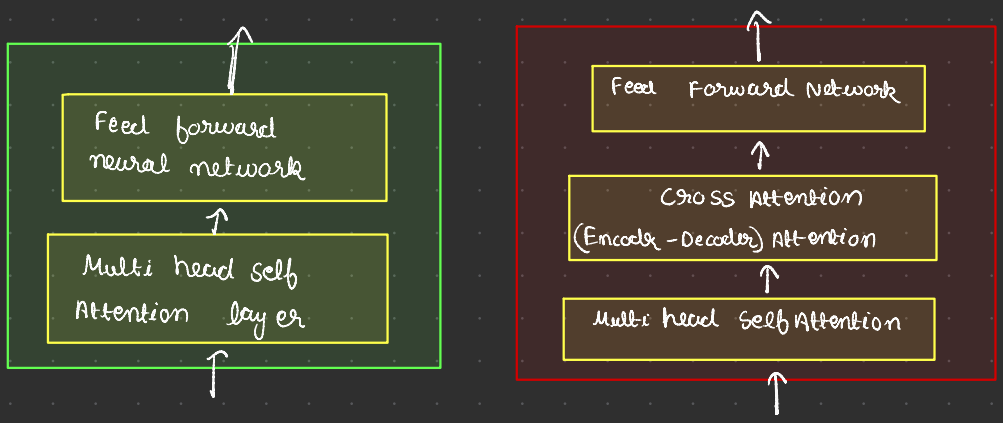
middle layers → Understanding the context

Higher layers → capture the Abstract meaning

Similar to CNN



A single encoder and Decoder Block



A single encoder cell

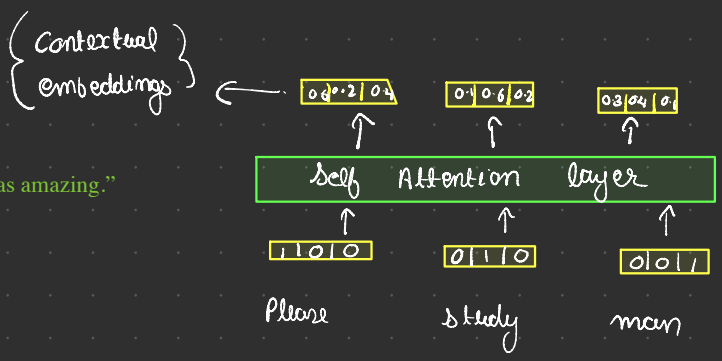
Now when we stack multiple encoders and Decoders layer and feed the data you will see the transformers in working.

Made with Goodnotes

Encoder

Multi head self attention layer

Multi-head self-attention is a mechanism that lets a model look at all words in a sentence simultaneously from multiple perspectives, so it can understand context and relationships more deeply before making a prediction.



Older models like RNNs struggled with long sentences.
Example:
"The food at the restaurant was terrible… but the **dessert** was amazing."

The word **"amazing"** relates to **dessert**, not food.
But they are far apart.

Self-attention fixes this by allowing:
Every word to directly talk to every other word.
Distance becomes irrelevant.
This was revolutionary.

Query

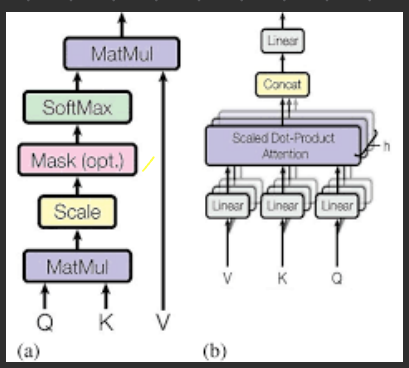
A Query represents what a word is trying to find in the rest of the sentence. You can think of it as a question the word is asking, such as "Which other words are important for my meaning?" When the model processes a token, it generates a Query vector that gets compared with the Key vectors of all other tokens. The stronger the match, the more attention that word receives. In simple terms, the Query is the search signal that helps a word gather the right context before forming a deeper understanding.

Key

A Key is like a label that describes what information a word contains and how relevant it might be to others. Every token produces a Key vector, and these keys are checked against incoming Queries to measure compatibility. If a Key closely aligns with a Query, the model interprets that word as important for the current context. Essentially, Keys help answer the question, "Do I have the information you are looking for?" This matching process is what allows the model to identify relationships between words, regardless of how far apart they appear in a sentence.

Value

A Value holds the actual information that will be shared once a strong Query–Key match is found. After calculating attention scores, the model does not pass along the Keys or Queries it combines the Value vectors based on how much attention each word deserves. This weighted blending creates a richer, contextual representation of the token. Put simply, if the Query is the question and the Key determines relevance, the Value is the knowledge that gets delivered, enabling the model to understand language in a more connected and meaningful way.



Made with Goodnotes

Step - 1 - Creating Q K V - Vectors

Please | Study | Man

[1 | 0] [0 | 2] [1 | 1]

Create - Query Key Value

Query = W_q W_K W_V

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

Calculating queries

Please vector • W_q

$$[1,0] \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$[1\times1 + 0\times0,\ 1\times0,\ 0\times1] = [1,0]$$

Please = [1,0]
Study = [0,2]
Man = [1,1]

Calculate Keys

Please vector • W_K

Please = [1,1]
Study = [0,2]
Man = [1,2]

Calculate values

Please vector • W_V

Please => [1,0]
Study => [2,2]
Man => [2,1]

Step - 2 - Raw Attention Score.

Score → $Q \times K^T$

Please = Q = [1,0]

Keys Keys

Please => [1,1]
Study => [0,2]
Man => [1,2]

$[1,0] \cdot [1,1] = 1$
$[1,0] \cdot [0,2] = 0$
$[1,0] \cdot [1,2] = 1$

Made with Goodnotes

Study Attention score

$[0,2] \cdot [1,1] = 2$

$[0,2] \cdot [0,2] = 4$

$[0,2] \cdot [1,2] = 4$

Man

Q

$[1,1] \cdot [1,1] = 2$

$[1,1] \cdot [0,2] = 2$

$[1,1] \cdot [1,2] = 3$

	Please	Study	man
Please	1	[0]	1
Study	2	4	[4]
Man	[2]	2	3

Relationship map of the sentence

Raw scores are just numbers

we have to turn Attention score to Attention weights

here we will convert the score to Probabilities → Softmax

Step-3 → Applying Softmax

	Please	Study	man
Please	1	[0]	1
Study	2	4	[4]
Man	[2]	2	3

Study

$[2,4,4] \rightarrow [0.06, 0.47, 0.47]$

Study gives → 6% Attention to Please

→ 47% Attention to itself

→ 47% " " man

Please

$[1 \quad 0 \quad 1]$

⇓

$[0.42, 0.16, 0.42]$

Man

$[2, 2, 3]$

⇓

$[0.21, 0.21, 0.58]$

Made with Goodnotes

Step - 4 → creating the context vector

Context vector = Attention weights x values

Please Study Mean

Study → [0.06, 0.47, 0.47]

Values ⇒ Please ⇒ [1,0]
Study ⇒ [2,2]
Mean ⇒ [2,1]

0.06 x [1,0] = [0.06, 0]
0.47 x [2,2] = [0.94, 0.94] +
0.47 x [2,1] = [0.94, 0.47] +

Contextual vector for Study ⇒ [1.94, 1.41]

This vector (Study) is influenced by another words

Study [0,2] → Self Attention Layer → [1.94, 1.41] → contextual vector

Final Summary

Step-1 → creating Q, K, V
Step-2 → Raw Attention Score
Step-3 → Softmax [Attention weights]
Step-4 → weighted sum values (context vector]

Embedding-512
Head - 8
64 64 64

Sentence
Grammer — Head-1 — QKV
meaning — Head-2
Structure — Head-3
etc

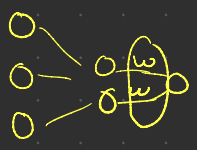
Made with Goodnotes

Now before going to feed forward network there are 3 things you Do.

① Concatinating the Head [1 64 | 64 | 64 | - - - - | 64]^8

512 vectore

② Linear Projection
(output weights)



Output = Concat(heads) × W_o

③ Layer Normalization

Z = LayerNorm(Output + Original input)

Norm ⇒ Stabilize the values

Residual ⇒ Keep the original info.

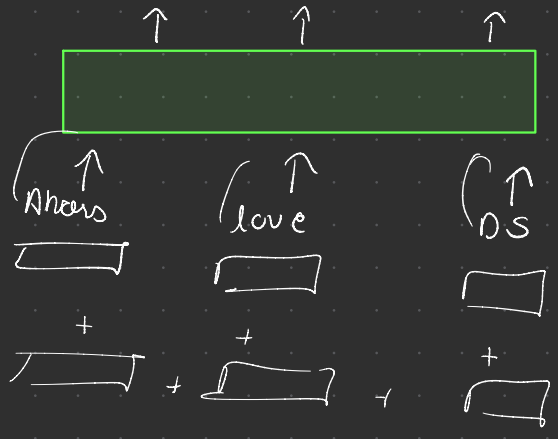
Vanishing Gradient.

↑↑

because of Positional encoding

because transformers works Parallelly

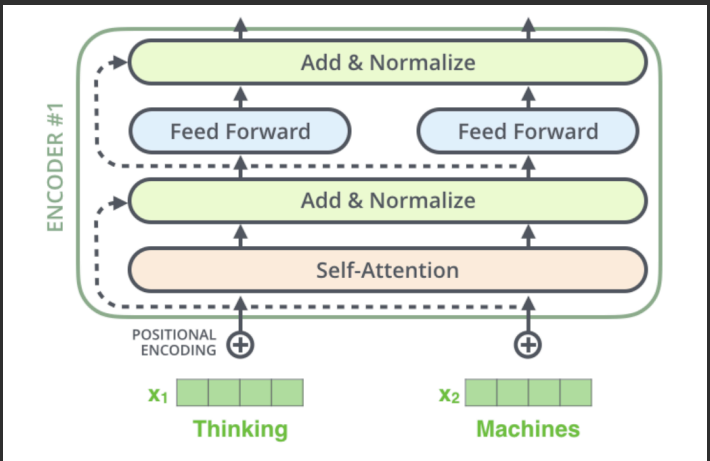
Akash	DS
love	love
DS	Akash



Made with Goodnotes

Feed forward neural network

The Feed Forward Neural Network (FFN) is the second main part of an encoder layer that comes after self-attention. Once self-attention helps each word understand its relationship with other words, the FFN processes each word individually to extract deeper patterns and meaning. It is simply a two-layer neural network (Linear → Activation → Linear) that first expands the embedding size (for example, 512 to 2048) to give the model more capacity to learn complex features, and then compresses it back to the original size. Unlike attention, there is no communication between words here the same network is applied separately to every token. In short, self-attention builds context, and the feed forward network strengthens that understanding by transforming it into a richer representation.



Multiple encoder layers are used to gradually deepen the model's understanding of the sentence. The first layer typically captures basic relationships between words, such as nearby context, while the next layers build on that information to detect more complex patterns like long-range dependencies and semantic meaning. Each layer refines the representation further, making it more contextual and abstract. Just like deep neural networks become more powerful with depth, stacking encoder layers allows the transformer to move from simple word-level understanding to a much richer comprehension of the entire sentence.

Made with Goodnotes

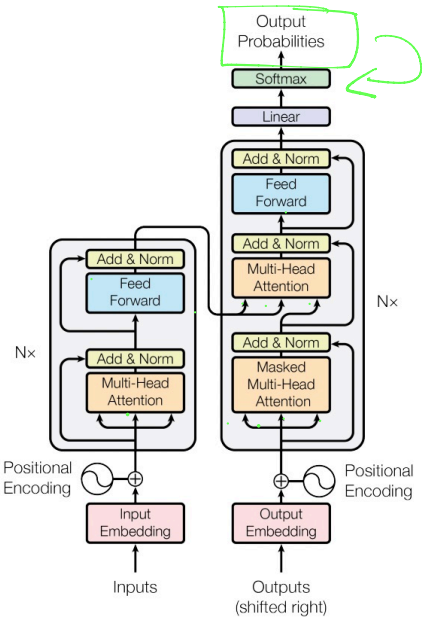


Figure 1: The Transformer - model architecture.

x

Please Study Man

y

कृपया पढ़ें मनुष्य

1. Masked multihead attention
2. Multi head Attention (Encoder, Decoder Attention)
3. Feed Forward NN.

Transformer

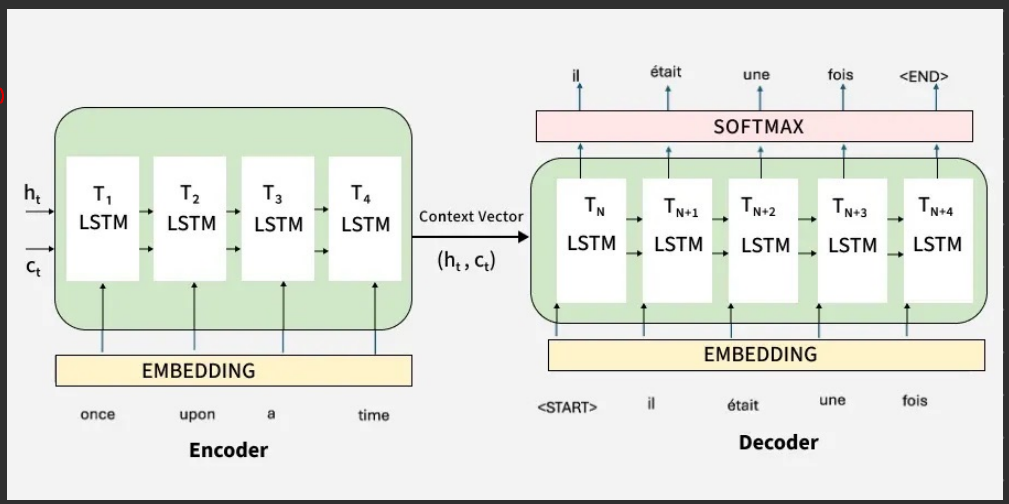
Training

Non Auto Regressive

this is because of masked multi head self Attention

Testing

Auto Regressive

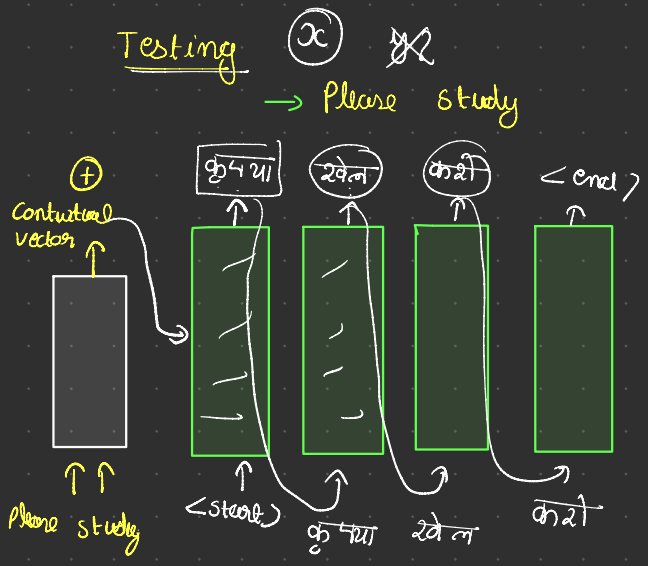


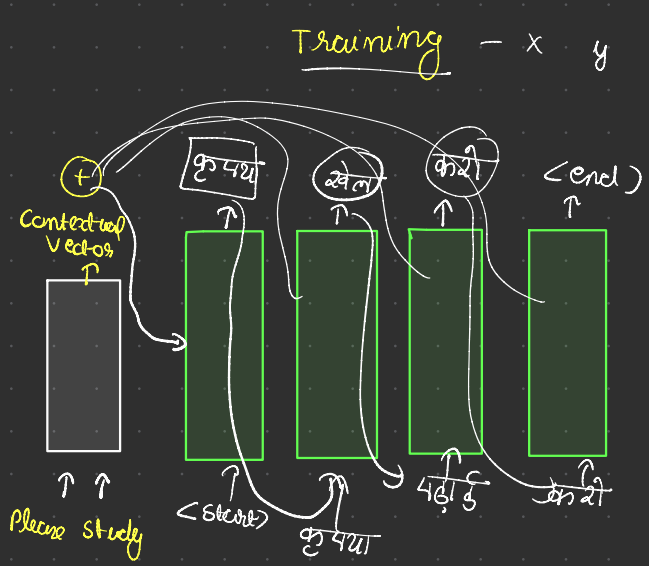
Made with Goodnotes

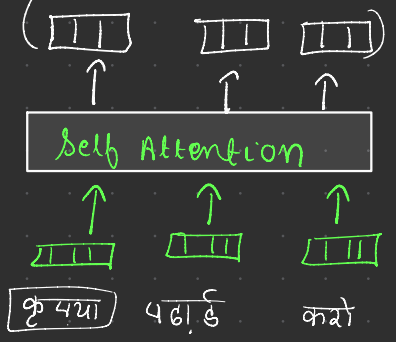
300w

English Sentence	Hindi Sentence
Please study.	कृपया पढ़ाई करो।
I am hungry.	मुझे भूख लगी है।
Where are you going?	तुम कहाँ जा रहे हो?

326







कृपया → [0.8, 0.1, 0.1]

[0.2, 0.6, 0.1]

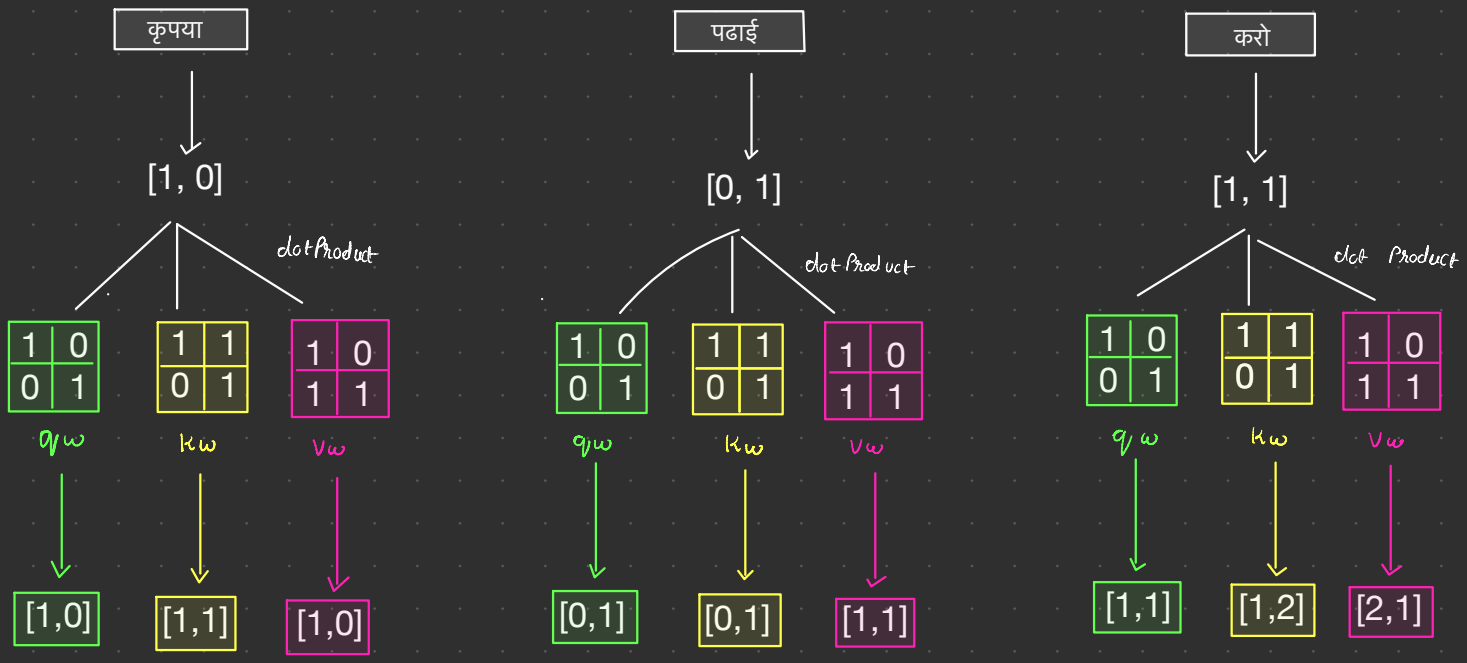
पढ़ाई → [0.4, 0.2, 0.4]

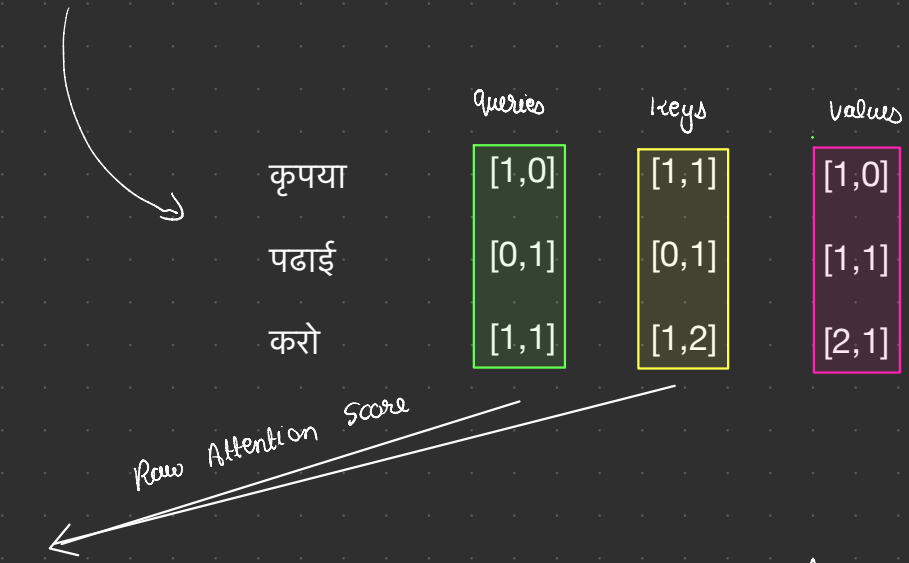
Cheating

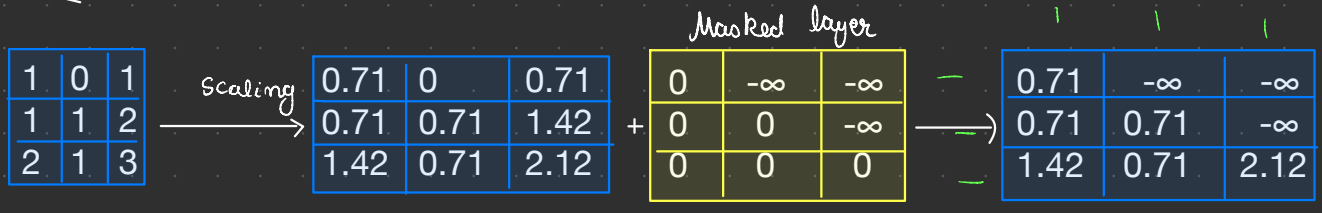
AR ✗
→ Slow
→ No Data leakage

NAR
→ fast
→ Data leakage

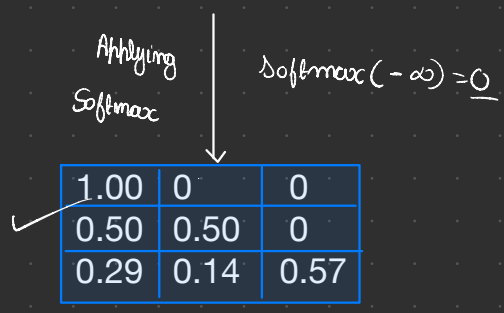
Made with Goodnotes







Raw Attention Score



कृपया → [1.00 , 0.00]

पढाई → [1.00 , 0.50]

करो → [1.57 , 0.71]

☆ → A word can only see itself and Past words
→ future words are blocked

Made with Goodnotes

Encoders

Please study man

Please — — —
Study — — —
man — — —

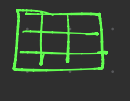
Q K V

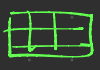
[ ]

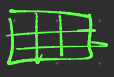
Decoders

कृपया पढ़ाई करो

Q K V

Cross Attention → Q – output cont. emb. –  × [ ] → (output)

K → input cont. emb –  × [ ] → (inp)

V → input cont emb –  × [ ] → (inp)

	Please	Study	man
कृपया	–	–	–
→ पढ़ाई	–	–	–
करो	–	–	–

Made with Goodnotes